



VulnHunterX

— LLM-VERIFIED SAST · HANDS-ON WORKSHOP

Find real bugs. Skip the false alarms.

Static analysis finds candidates — an LLM decides which are real.

64 custom CodeQL queries

103 custom Semgrep rules

8 languages · 5 profiles

60-minute workshop · for developers · CodeQL + Semgrep + LLM triage

Agenda — 60 minutes

VulnHunterX

1 • The problem

Why SAST drowns you in false positives · 5 min

2 • What VulnHunterX is

The one-sentence pitch · 5 min

3 • How it works

Pipeline + the multi-turn loop · 12 min

4 • Toolchain & install

CodeQL, Semgrep, tree-sitter · 10 min

5 • Windows?

What runs natively, what needs WSL2 · 5 min

6 • Workshop

Live demo — run it together · 10 min

7 • Exercises

Hands-on + homework · 5 min

Wrap-up

Recap & references · 3 min



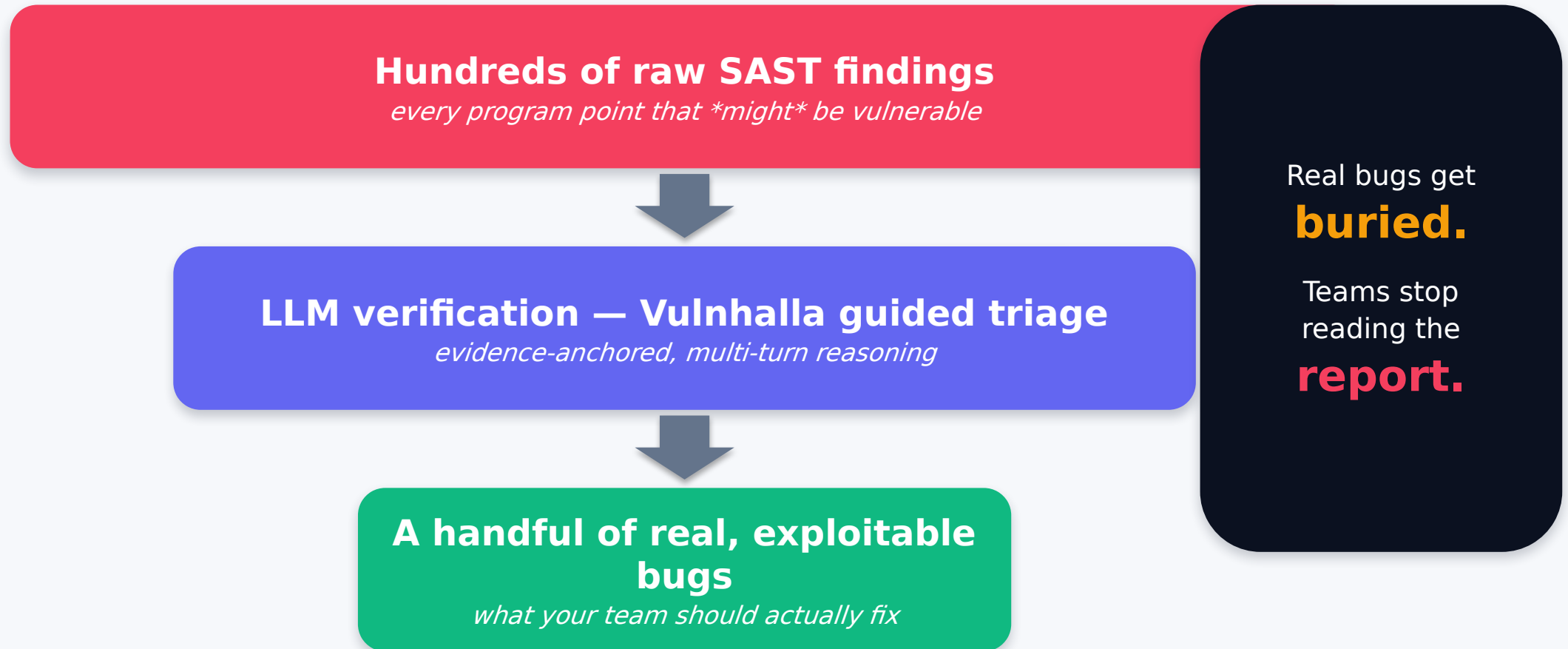
PART 1

The problem



From noise to signal

SAST over-approximates by design — the cost is human triage, not the scan.





PART 2

What VulnHunterX is



A Python framework that pairs **SAST engines** with **multi-turn LLM verification** to suppress false positives — the **Vulnhalla** guided-question methodology.

Cite evidence

the LLM must point at real code

Pull context

callers, structs, globals on demand

Reason in turns

not pattern-match one snippet

Key features

VulnHunterX

8 languages

C, C++, Python, JavaScript, PHP, Java, Go, C#

3 SAST engines

CodeQL · Semgrep · OpenGrep (--tool ...)

5 rule profiles

standard → extended → maximum → ... → full

LLM providers

OpenAI · Anthropic · Ollama — via LiteLLM

Multi-turn verification

context expansion → verdict + confidence

Flexible inputs

Git URL · local dir · batch repos.yaml

Reports

Markdown EN/VI — summary + per-finding detail

Fuzzing (C/C++)

libFuzzer / Atheris / Jazzer + crash triage

 PART 3

How it works



The pipeline — 5 steps, 1 command

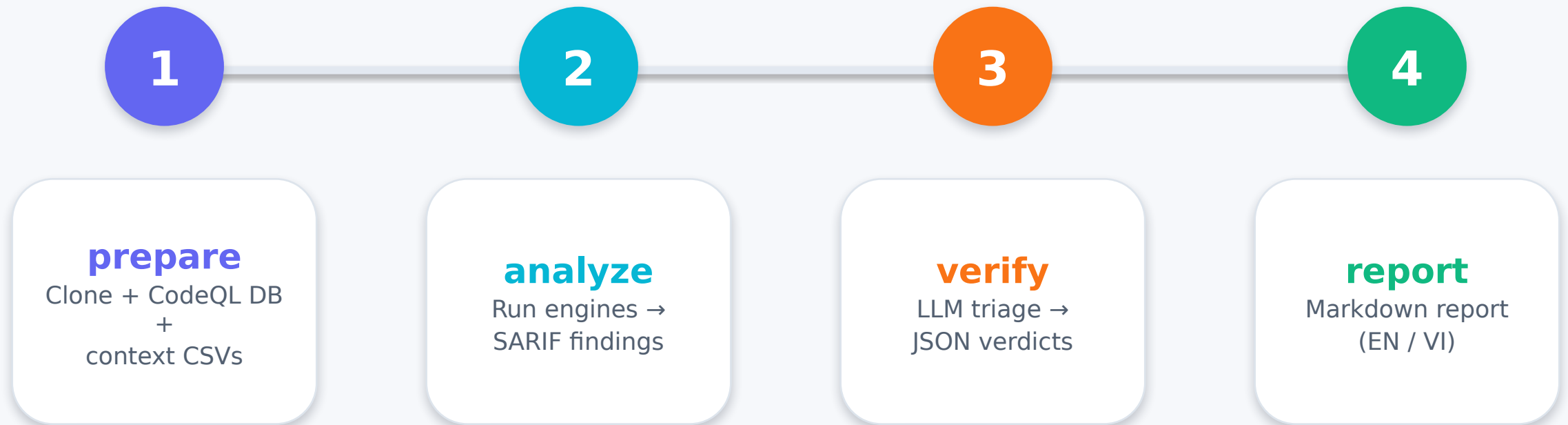
VulnHunterX



+ 4 optional fuzz stages (C/C++ only)



The 4 core stages



The Vulnhalla multi-turn loop

VulnHunterX



Loop up to **max_iterations (default 3)** until the LLM commits.

What a verdict looks like

Finding #12 · cpp/use-after-free

TRUE
POSITIVE

Confidence

High · 0.86

Reasoning

Tainted length from recv() reaches memcpy() with no bounds check; attacker-controlled. Confirmed by fetching caller parse_packet(), which frees buf before the copy. Reachable on the error path.

2 turns

CWE-416

verdict TP / FP / Needs More Data

confidence High / Medium / Low + score

reasoning evidence-anchored, cites code

iterations turns the LLM took

data_flow source→sink trace (CodeQL)

Rules & coverage

VulnHunterX

Counted from config/ at build time — not copied from the README.

64

custom CodeQL queries

cpp 16 · csharp 5 · go 8 · java 10 ·
javascript 14 · python 11

103

custom Semgrep rules

cpp 4 · csharp 14 · go 19 · java 16 ·
javascript 14 · php 14 · python 22

~366

**guided-question sets · 8
banks**

rule-specific checklists

5 rule profiles standard → extended → maximum → extended-registry → full
124-entry CWE routing map routes findings to the right guided questions
~5-10× more rules from --profile standard to --profile full

 PART 4

The toolchain & installation



The four tools — what each does

VulnHunterX

CodeQL

Semantic, database-driven taint analysis — the deepest engine. Needs a DB build.

core

Semgrep

Fast pattern-based rules across many languages. No DB needed.

breadth

OpenGrep

Semgrep-compatible fork — same rules, drop-in alternative.

alt

tree-sitter

Syntactic context extraction — pip-installed fallback when there's no CodeQL DB.

fallback

Install VulnHunterX



```
# Prerequisites: Python 3.12+, CodeQL CLI 2.15+, an LLM provider key  
git clone https://github.com/vinsoc-cyber/VulnHunterX.git && cd VulnHunterX  
  
uv venv --python python3.12 .venv && source .venv/bin/activate  
uv pip install -e ".[dev]" # plain python3.12 -m venv + pip also works  
  
cp env.example .env # add OPENAI / ANTHROPIC / OLLAMA keys  
vuln-hunter-x check-env # verify the whole toolchain is green
```

Configuration you need to set

```
                                # .env - provider keys + optional tool paths
LLM_PROVIDER=openai             # openai | anthropic | ollama
LLM_MODEL=gpt-4o
OPENAI_API_KEY=sk-...           # or ANTHROPIC_API_KEY / OLLAMA_API_BASE
CODEQL_PATH=/path/to/codeql     # only if codeql isn't on your PATH
SEMGREP_PATH=/path/to/semgrep   # only if semgrep isn't on your PATH

# config/confirm_findings.yaml - verification tuning
provider: openai  model: gpt-4o  temperature: 0.2  max_iterations: 3

# config/repos.yaml - batch targets (name, url, language, build_command)
```



PART 5

Does it run on Windows?



Windows compatibility

VulnHunterX

VulnHunterX core

POSIX build scripts break C/C++ build automation natively

Partial

Semgrep

Use Docker or WSL2 for reliable runs

Limited

tree-sitter

Pure-Python wheels — works everywhere

Full

LLM providers

HTTP-based — no platform dependency

Full

CodeQL CLI

Official win64 binary; build cmd must be Windows-friendly

Yes

OpenGrep

Same as Semgrep — check releases / use WSL2

Limited

Fuzz stages 5-8

Need clang ASan/UBSan — Linux/macOS only

No

Recommendation for Windows users

Run VulnHunterX inside **WSL2** (Ubuntu) or a Linux **Docker** container.

Full toolchain — Semgrep, CodeQL, fuzzing — with no caveats.

Native Windows is fine for: editing code · reading reports · calling the LLM

Linux / macOS users: install natively — see Part 4

Windows users: wsl --install, then follow the same steps inside WSL2

PART 6

Workshop — let's run it



Three ways to start a scan

1) Friendliest: a guided wizard (prompts for everything, live-tests your LLM)
vuln-hunter-x interactive

2) One-shot: the whole pipeline in a single command
vuln-hunter-x scan --url https://github.com/org/app.git \
--lang python --profile extended --limit 5

3) Stage by stage (full control)
vuln-hunter-x prepare --repo pyyaml
vuln-hunter-x analyze --repo pyyaml --profile extended
vuln-hunter-x verify --repo pyyaml --limit 5
vuln-hunter-x report --repo pyyaml --lang python

The fastest demo: an example pipeline

```
# real-world repo + vulnerable repo, full
# pipeline on both
python examples/pipeline_python.py

# smoke-test without spending LLM calls:
python examples/pipeline_python.py --dry-run

cat
output/python/pyyaml/verification_results/report
.md
```

PyYAML — *real-world*
mostly clean → false positives
suppressed

dvpwa — *vulnerable*
surfaces real true positives

The whole value proposition in one screen:
the contrast between a clean real-world repo and a vulnerable one.
Per-language scripts exist for all 8 languages.

PART 7

Exercises



In-session exercises

VulnHunterX

5 min

E1 · Green toolchain

vuln-hunter-x check-env until every engine + provider is OK. (Windows: in WSL2/Docker.)

core

E2 · First scan

pipeline_python.py --dry-run, then a real --limit 3 run; open report.md, find 1 TP + 1 FP.

loop

E3 · Iterations matter

Re-run verify --max-iterations 5; compare verdicts/confidence vs the default 3.

explore

E4 · Another language

Run examples/pipeline_go.py (or java/php) and read its report.

do this

H1 · Scan your own code

`vuln-hunter-x scan --local-path <repo> --lang <lang> --profile full`

H2 · Compare providers

Verify the same SARIF with `--provider openai` vs `ollama`; note where verdicts differ.

H3 · Write a custom rule

Add to `semgrep-custom/<lang>.yaml` (metadata.cwe), run `--profile full`, then `audit_rule_coverage.py --fail-on-gaps`.

optional

H4 · Stretch (Linux/macOS)

Fuzz a C target: `build-sanitized` → `fuzz-run` on `libucl`.

Recap & references

SAST over-approximates: VulnHunterX adds an LLM triage stage, with evidence

Pipeline: prepare → analyze → verify → report

64 CodeQL + 103 Semgrep rules: 8 languages · 5 profiles

Windows: use WSL2 / Docker for the full toolchain

Start here → [vuln-hunter-x interactive](#)

References: Vulnhalla (CyberArk) · CodeQL docs · Semgrep docs · SARIF spec · repo README + config/RULES.md